CO634 Cryptography

From Alphabets to Binary

lectures by Carlos A. Perez-Delgado

School of Computing,
University of Kent

©2019

Outline

1. From Alphabets to Binary
   - MathsBite: Powers Modulo n
   - More on Exclusive-OR
   - The One-Time Pad in Binary
   - Unconditional and Computational Security

Outline

1. From Alphabets to Binary
   - MathsBite: Powers Modulo n
   - More on Exclusive-OR
   - The One-Time Pad in Binary
   - Unconditional and Computational Security

MathsBite: Powers modulo n

In ordinary arithmetic, if a is an integer and j is a positive integer, we use the notation a^j to denote a multiplied by itself j times:

$$\begin{aligned} a^1 &= a \\ a^2 &= a \times a \\ a^3 &= a \times a \times a \\ a^4 &= a \times a \times a \times a \\ &\vdots \end{aligned}$$

a is called the **base**, and j the **exponent**.

We can define a similar operation in the world of arithmetic modulo n: multiply a by itself j times, but then reduce the result modulo n (i.e. take the remainder when a^j is divided by n).

MathsBite: Powers modulo n

In ordinary arithmetic, if a is an integer and j is a positive integer, we use the notation a^j to denote a multiplied by itself j times:

$$\begin{aligned} a^1 &= a \\ a^2 &= a \times a \\ a^3 &= a \times a \times a \\ a^4 &= a \times a \times a \times a \\ &\vdots \end{aligned}$$

a is called the **base**, and j the **exponent**.

We can define a similar operation in the world of arithmetic modulo n: multiply a by itself j times, but then reduce the result modulo n (i.e. take the remainder when a^j is divided by n).

MathsBite: Powers Modulo 9

For example, let's take a look at powers modulo 9:

$\mathbf{i = a^1}$	$\mathbf{a^2}$	$\mathbf{a^3}$	$\mathbf{a^4}$	$\mathbf{a^5}$	$\mathbf{a^6}$	$\mathbf{a^7}$	$\mathbf{a^8}$	$\mathbf{a^9}$	$\mathbf{a^{10}}$
0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1
2	4	8	7	5	1	2	4	8	7
3	0	0	0	0	0	0	0	0	0
4	7	1	4	7	1	4	7	1	4
5	7	8	4	2	1	5	7	8	4
6	0	0	0	0	0	0	0	0	0
7	4	1	7	4	1	7	4	1	7
8	1	8	1	8	1	8	1	8	1

MathsBite: Powers Modulo 7

$\mathbf{i} = \mathbf{a}^1$	$\mathbf{a}^2$	$\mathbf{a}^3$	$\mathbf{a}^4$	$\mathbf{a}^5$	$\mathbf{a}^6$	$\mathbf{a}^7$
0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	4	1	2	4	1	2
3	2	6	4	5	1	3
4	2	1	4	2	1	4
5	4	6	2	3	1	5
6	1	6	1	6	1	6

Prime Numbers

A **prime number** is an integer greater than one that is not divisible by any integer apart from itself and one.

The first few prime numbers are 2, 3, 5, 7, 11, 13, 17, 19, 23 and 29.

Stallings [2006] Table 8.1 shows all the prime numbers up to 2000 (with one mistake!—see the online errata).

MathsBite: Theorem 3 (Fermat)

If a is any integer and p is a prime number, then:

$$a^{p-1} \textbf{ mod } p = \begin{cases} 0 & \textbf{if } a \textbf{ mod } p = 0 \\ 1 & \textbf{otherwise} \end{cases}$$

For a proof, see Stallings [2006] Sec. 8.2.

Corollary: If a is any integer and p is a prime number, then:

$$a^p \bmod p = a \bmod p$$

MathsBite: Theorem 3 (Fermat)

If a is any integer and p is a prime number, then:

$$a^{p-1} \textbf{ mod } p = \begin{cases} 0 & \textbf{if } a \textbf{ mod } p = 0 \\ 1 & \textbf{otherwise} \end{cases}$$

For a proof, see Stallings [2006] Sec. 8.2.

Corollary: If a is any integer and p is a prime number, then:

$$a^p \bmod p = a \bmod p$$

Outline

1. From Alphabets to Binary
   - MathsBite: Powers Modulo n
   - More on Exclusive-OR
   - The One-Time Pad in Binary
   - Unconditional and Computational Security

XOR Applied to Bit Patterns

In the last lecture we met the exclusive-OR (XOR) operation as applied to single 0/1 bits. We can also apply it as an operation on (equal-lengthed) bit-strings, e.g. bytes. We simply apply the $\oplus$ operation to each bit in turn. For example:

$$\begin{array}{cccccccc} & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ \oplus & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ \hline & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \end{array}$$

Question: Evaluate

$$\begin{array}{cccccccc} & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ \oplus & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ \hline \end{array}$$

XOR Applied to Bit Patterns

In the last lecture we met the exclusive-OR (XOR) operation as applied to single 0/1 bits. We can also apply it as an operation on (equal-lengthed) bit-strings, e.g. bytes. We simply apply the $\oplus$ operation to each bit in turn. For example:

$$\begin{array}{cccccccc} & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ \oplus & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ \hline & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \end{array}$$

Question: Evaluate

$$\begin{array}{cccccccc} & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ \oplus & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ \hline \end{array}$$

Properties of XOR

If A and B are bit-strings and

$$A \oplus B = C$$

then:

- $B \oplus A = C$ (fairly obviously). But also:
- $A = B \oplus C$, and
- $B = A \oplus C$.

So, given any two of A, B or C, we can XOR them together to get the third.

This fact is used in some RAID arrays of three disks: roughly speaking, each sector on the third disk holds the XOR of the data in the corresponding sectors on the first two disks.

Properties of XOR

If A and B are bit-strings and

$$A \oplus B = C$$

then:

- $B \oplus A = C$ (fairly obviously). But also:
- $A = B \oplus C$, and
- $B = A \oplus C$.

So, given any two of A, B or C, we can XOR them together to get the third.

This fact is used in some RAID arrays of three disks: roughly speaking, each sector on the third disk holds the XOR of the data in the corresponding sectors on the first two disks.

Properties of XOR

If A and B are bit-strings and

$$A \oplus B = C$$

then:

- $B \oplus A = C$ (fairly obviously). But also:
- $A = B \oplus C$, and
- $B = A \oplus C$.

So, given any two of A, B or C, we can XOR them together to get the third.

This fact is used in some RAID arrays of three disks: roughly speaking, each sector on the third disk holds the XOR of the data in the corresponding sectors on the first two disks.

Properties of XOR

If A and B are bit-strings and

$$A \oplus B = C$$

then:

- $B \oplus A = C$ (fairly obviously). But also:
- $A = B \oplus C$, and
- $B = A \oplus C$.

So, given any two of A, B or C, we can XOR them together to get the third.

This fact is used in some RAID arrays of three disks: roughly speaking, each sector on the third disk holds the XOR of the data in the corresponding sectors on the first two disks.

Outline

1. From Alphabets to Binary
   - MathsBite: Powers Modulo n
   - More on Exclusive-OR
   - The One-Time Pad in Binary
   - Unconditional and Computational Security

The One-Time Pad Revisited

Moving to binary

All data on computers is stored as a sequence of 0/1 bits, usually further aggregated into bytes, blocks etc., so in computer cryptography we naturally think of encrypting the binary data directly, rather than working in terms of alphabetic letters etc.

Let's think how the one-time pad would work with an 'alphabet' consisting just of the 'letters' 0 and 1.

The One-Time Pad Revisited

Suppose this is the start of our plaintext (*Bredon Hill* by A. E. Housman). The first row shows the characters of the poem, and below that their binary representation (using ASCII encoding).

```
I       n                 s       u       m       m       e       r
01001001011011100010000001110011011101010110110101101101101100101011100 10
```

Our key now needs to be simply a random sequence of bits, for example:

```
1011010111100010010101000100101010001111110001100001101111011010111101 00
```

To encrypt the message, using the usual rule for Vigenère ciphers, we add each ‘letter’ of the plaintext to the corresponding ‘letter’ of the key, and reduce the result modulo the number of letters in the alphabet, i.e. 2. In other words we simply XOR the plaintext with the key, getting:

```
1111110010001100011101000011100111111010101010110111011010111111100001 10
```

How do we decrypt the message?

The One-Time Pad Revisited

Suppose this is the start of our plaintext (*Bredon Hill* by A. E. Housman). The first row shows the characters of the poem, and below that their binary representation (using ASCII encoding).

```
I       n                 s       u       m       m       e       r
01001001011011100010000001110011011101010110110101101101101100101011100 10
```

Our key now needs to be simply a random sequence of bits, for example:

```
1011010111100010010101000100101010001111110001100001101111011010111101 00
```

To encrypt the message, using the usual rule for Vigenère ciphers, we add each 'letter' of the plaintext to the corresponding 'letter' of the key, and reduce the result modulo the number of letters in the alphabet, i.e. 2. In other words we simply XOR the plaintext with the key, getting:

```
1111110010001100011101000011100111111010101010110111011010111111100001 10
```

How do we decrypt the message?

The One-Time Pad Revisited

Suppose this is the start of our plaintext (*Bredon Hill* by A. E. Housman). The first row shows the characters of the poem, and below that their binary representation (using ASCII encoding).

```
I       n                 s       u       m       m       e       r
01001001011011100010000001110011011101010110110101101101100101011100 10
```

Our key now needs to be simply a random sequence of bits, for example:

```
10110101111000100101010001001010100011111100011000011011110110101111010 0
```

To encrypt the message, using the usual rule for Vigenère ciphers, we add each 'letter' of the plaintext to the corresponding 'letter' of the key, and reduce the result modulo the number of letters in the alphabet, i.e. 2. In other words we simply XOR the plaintext with the key, getting:

```
11111100100011000111010000111001111110101010101101110110101111111000011 0
```

How do we decrypt the message?

The One-Time Pad Revisited

Suppose this is the start of our plaintext (*Bredon Hill* by A. E. Housman). The first row shows the characters of the poem, and below that their binary representation (using ASCII encoding).

```
I       n                 s       u       m       m       e       r
01001001011011100010000001110011011101010110110101101101101100101011100 10
```

Our key now needs to be simply a random sequence of bits, for example:

```
10110101111000100101010001001010100011111100011000011011110110101111010 0
```

To encrypt the message, using the usual rule for Vigenère ciphers, we add each 'letter' of the plaintext to the corresponding 'letter' of the key, and reduce the result modulo the number of letters in the alphabet, i.e. 2. In other words we simply XOR the plaintext with the key, getting:

```
11111100100011000111010000111001111110101010101101110110101111110000110
```

How do we decrypt the message?

The One-Time Pad Revisited

Suppose this is the start of our plaintext (*Bredon Hill* by A. E. Housman). The first row shows the characters of the poem, and below that their binary representation (using ASCII encoding).

```
I       n               s       u       m       m       e       r
01001001011011100010000001110011011101010110110101101101101100101011100 10
```

Our key now needs to be simply a random sequence of bits, for example:

```
10110101111000100101010001001010100011111100011000011011110110101111010 0
```

To encrypt the message, using the usual rule for Vigenère ciphers, we add each ‘letter’ of the plaintext to the corresponding ‘letter’ of the key, and reduce the result modulo the number of letters in the alphabet, i.e. 2. In other words we simply XOR the plaintext with the key, getting:

```
11111100100011000111010000111001111110101010101101110110101111110000110
```

How do we decrypt the message?

The One-Time Pad Revisited

Randomness of ciphertext

Take a look at the ciphertext from the previous slide:

```
1111110010001100011101000011100111111010101010110111011010111111110000110
```

This looks pretty much like a random sequence of bits ...

... and not just to casual observation. Provided that the key is truly random, the ciphertext is *statistically indistinguishable* from a random sequence of bits (i.e. a sequence in which each bit is 1 with probability $\frac{1}{2}$, and is statistically independent of all preceding bits).

This makes it impossible for cryptanalysts to get a handle on the code. Frequency analysts, eat your heart out!

The One-Time Pad Revisited

Randomness of ciphertext

Take a look at the ciphertext from the previous slide:

```
1111110010001100011101000011100111111010101010110111011010111111110000110
```

This looks pretty much like a random sequence of bits ...

... and not just to casual observation. Provided that the key is truly random, the ciphertext is *statistically indistinguishable* from a random sequence of bits (i.e. a sequence in which each bit is 1 with probability $\frac{1}{2}$, and is statistically independent of all preceding bits).

This makes it impossible for cryptanalysts to get a handle on the code. Frequency analysts, eat your heart out!

The One-Time Pad Revisited

Randomness of ciphertext

Take a look at the ciphertext from the previous slide:

```
1111110010001100011101000011100111111010101010110111011010111111110000110
```

This looks pretty much like a random sequence of bits ...

... and not just to casual observation. Provided that the key is truly random, the ciphertext is *statistically indistinguishable* from a random sequence of bits (i.e. a sequence in which each bit is 1 with probability $\frac{1}{2}$, and is statistically independent of all preceding bits).

This makes it impossible for cryptanalysts to get a handle on the code. Frequency analysts, eat your heart out!

Problems with the One-Time Pad

As we've seen, a one-time pad is effectively unbreakable provided the key is secure. The big problem is how to distribute the key securely.

For example, suppose that a company's depot sends 100 MB of commercially sensitive data to head office each day over the internet, using a one-time pad.

Head office won't be able to read the data unless it has the same key as the depot, so somehow an average of 100 MB of random bits must be transmitted from head office to the depot (or *vice versa*) each day. How do you do that?

A secondary issue is that it is not trivial to generate long sequences of genuinely random data.

Problems with the One-Time Pad

As we've seen, a one-time pad is effectively unbreakable provided the key is secure. The big problem is how to distribute the key securely.

For example, suppose that a company's depot sends 100 MB of commercially sensitive data to head office each day over the internet, using a one-time pad.

Head office won't be able to read the data unless it has the same key as the depot, so somehow an average of 100 MB of random bits must be transmitted from head office to the depot (or *vice versa*) each day. How do you do that?

A secondary issue is that it is not trivial to generate long sequences of genuinely random data.

Problems with the One-Time Pad

As we've seen, a one-time pad is effectively unbreakable provided the key is secure. The big problem is how to distribute the key securely.

For example, suppose that a company's depot sends 100 MB of commercially sensitive data to head office each day over the internet, using a one-time pad.

Head office won't be able to read the data unless it has the same key as the depot, so somehow an average of 100 MB of random bits must be transmitted from head office to the depot (or *vice versa*) each day. How do you do that?

A secondary issue is that it is not trivial to generate long sequences of genuinely random data.

Problems with the One-Time Pad

As we've seen, a one-time pad is effectively unbreakable provided the key is secure. The big problem is how to distribute the key securely.

For example, suppose that a company's depot sends 100 MB of commercially sensitive data to head office each day over the internet, using a one-time pad.

Head office won't be able to read the data unless it has the same key as the depot, so somehow an average of 100 MB of random bits must be transmitted from head office to the depot (or *vice versa*) each day. How do you do that?

A secondary issue is that it is not trivial to generate long sequences of genuinely random data.

Outline

1. From Alphabets to Binary
   - MathsBite: Powers Modulo n
   - More on Exclusive-OR
   - The One-Time Pad in Binary
   - Unconditional and Computational Security

Unconditional Security

An encryption scheme is said to be **unconditionally secure** if the ciphertext it generates provides an adversary with no information about the corresponding plaintext, even if the adversary has unlimited computing resources, and access to unlimited amounts of ciphertext. (cf. Menezes *et al.* [1997] Sec. 1.13.3; Stallings [2006] uses a weaker formulation.)

In the 1940s Claude Shannon established that the only unconditionally secure encryption scheme is a one-time pad, or something very like it: in particular, the key must contain at least as many bits as there are bits in the plaintext, and these bits must be randomly chosen.

Unconditional Security

An encryption scheme is said to be **unconditionally secure** if the ciphertext it generates provides an adversary with no information about the corresponding plaintext, even if the adversary has unlimited computing resources, and access to unlimited amounts of ciphertext. (cf. Menezes *et al.* [1997] Sec. 1.13.3; Stallings [2006] uses a weaker formulation.)

In the 1940s Claude Shannon established that the only unconditionally secure encryption scheme is a one-time pad, or something very like it: in particular, the key must contain at least as many bits as there are bits in the plaintext, and these bits must be randomly chosen.

Computational Security

An encryption scheme is said to be **computationally secure** if either:

- The computational cost of breaking the cipher exceeds the value of the encrypted information, or
- The time required to break the cipher exceeds the useful lifetime of the encrypted information.

(Stallings [2006] p. 34).

Notice that a cipher that is computationally secure now may not be secure in the future, owing (*a*) to advances in computing technology, and (*b*) to theoretical advances in cryptanalysis.

Computational Security

An encryption scheme is said to be **computationally secure** if either:

- The computational cost of breaking the cipher exceeds the value of the encrypted information, or
- The time required to break the cipher exceeds the useful lifetime of the encrypted information.

(Stallings [2006] p. 34).

Notice that a cipher that is computationally secure now may not be secure in the future, owing (*a*) to advances in computing technology, and (*b*) to theoretical advances in cryptanalysis.

Computational Security

An encryption scheme is said to be **computationally secure** if either:

- The computational cost of breaking the cipher exceeds the value of the encrypted information, or
- The time required to break the cipher exceeds the useful lifetime of the encrypted information.

(Stallings [2006] p. 34).

Notice that a cipher that is computationally secure now may not be secure in the future, owing (*a*) to advances in computing technology, and (*b*) to theoretical advances in cryptanalysis.

Recap

1. From Alphabets to Binary
   - MathsBite: Powers Modulo n
   - More on Exclusive-OR
   - The One-Time Pad in Binary
   - Unconditional and Computational Security